CLAUDE CODE
COCKPIT
INTERNAL BUILD

STATUS BRIEF

CC COCKPIT · BUILD

CC Cockpit — Build Status
Spec Final · Both Spikes Green

Where the Claude Code cockpit stands after the dev-team run: a finalized build spec, both v1 risk-gates proven, and exactly what you need to do to keep going.

Built by the **lead-orchestrated dev-team** · product-owner · designer · dev · frontend · qa

Repo: ~/Workflows/cc-cockpit/ (standalone git, @ 12dd232)
Spec: docs/plans/2026-06-18-cc-cockpit-spec.md + 5 sections (prd / design / backend / frontend / qa)
Stack: Tauri (Rust) + SolidJS + xterm.js over tmux control-mode; macOS, Claude Max (no API-key infra)
Prepared: 18 June 2026

2/2	15	13/13	17	3
SPIKES GREEN	FEATURES SPECCED	D3 PARSER TESTS	RISKS LOGGED	CORPUS FIXES

PREPARED 18 JUNE 2026 · INTERNAL TOOLING STATUS

At a Glance

The spec is FINAL and both v1 risk-gates passed GREEN. The backbone build is unblocked — only two manual confirmation passes stand between here and v1.

The two hardest unknowns are de-risked with real, tested code: live per-pane state reading (D6) and rendering a real Claude TUI through tmux control-mode (D3). Nothing is running on the agent side now; the next moves are yours.

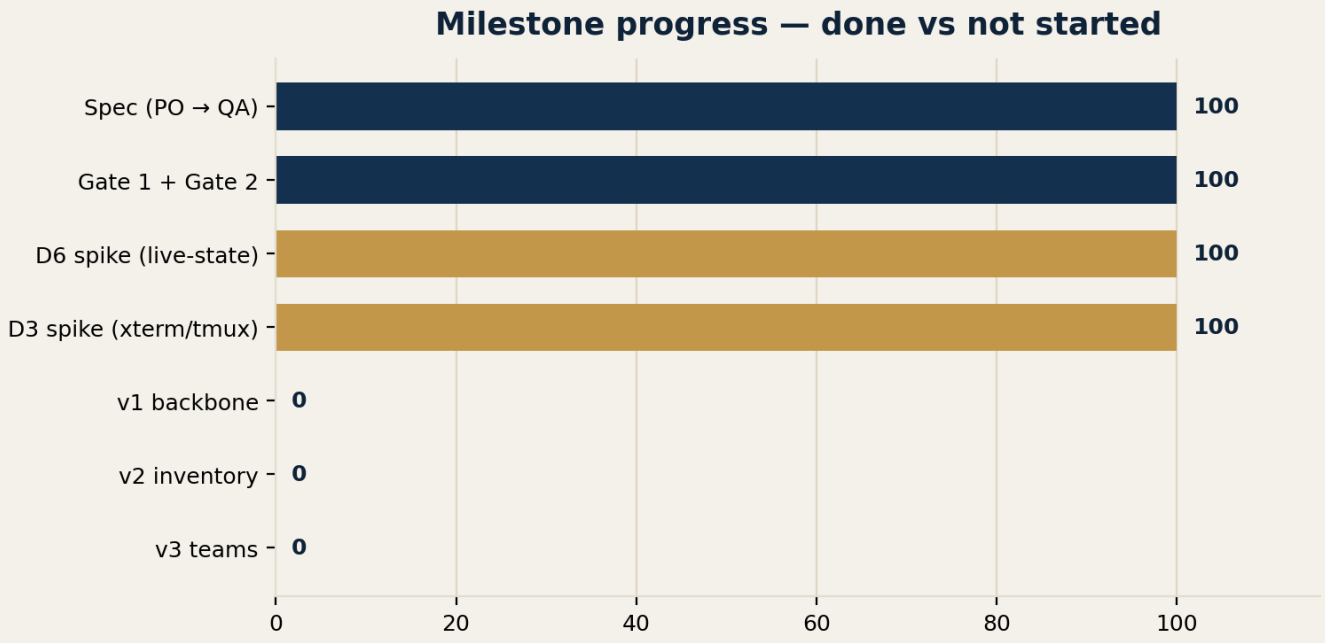
GREEN	GREEN	3	0
D6 SPIKE	D3 SPIKE	CORPUS FIXES	LINES OF V1
Live agent-state parser scored 4/4 on real Claude output captures; hook shim + pane mapping work.	tmux control-mode parser passed 13/13 tests vs real frames; Tauri + SolidJS both build clean.	Reading the real machine caught 3 spec-vs-reality gaps before any code was written.	Backbone not started — it waits behind the two spike gates, both now passed.

Build order — what ships when

1. **v1 — Multiplexing backbone (P1-F1…F6)** — Tier 1
   Tabs + tiled panes over tmux; launch / switch / kill / take-over CC sessions; per-pane status; attention routing. Daily-driver value, lowest risk. Builds behind the two passed spikes.
2. **v2 — Inventory mission-control (P2-F1…F5)** — Tier 2
   One panel to browse / enable / disable / configure skills + subagents + plugins + MCP across global + project. The clearest whitespace — no tool nails it. Gated by the config-safety test suite.
3. **v3 — Lead-orchestrated teams (P3-F1…F5)** — Tier 3
   Saveable team templates + a file mailbox a lead routes work through. Second whitespace gap. Most ambitious; needs v1 + benefits from v2.

What's done

The dev-team ran end-to-end as a lead-orchestrated team (star topology): product-owner decomposed the work, two human gates locked scope and stack, then designer + backend + frontend designed in parallel, QA wrote the test + risk plan, and two feasibility spikes proved the riskiest unknowns.



- **Spec is FINAL** — `docs/plans/2026-06-18-cc-cockpit-spec.md` plus five companion sections (PRD, design, backend, frontend, QA). 15 features with testable acceptance criteria, a 17-item risk register, a config-safety test suite, and a per-feature native-overlap go/no-go gate.
- **Repo scaffolded** — `~/Workflows/cc-cockpit/` , standalone git at commit `12dd232` . Rust 1.96 installed.
- **Both v1 spike gates GREEN**, with real tested code in `cc-cockpit/spikes/{d6,d3}/` .

> ✓ **WHY THIS IS SOLID GROUND**
>
> The two spikes weren't paper exercises — they ran against the real machine. D6's parser was validated on **live Claude output captures**; D3's control-mode parser passed **13 tests against real `tmux -CC` protocol frames** (re-verified independently). The two things most likely to sink the project are now proven.

THREE CORPUS CORRECTIONS (CAUGHT BEFORE CODING)

Reading the real machine instead of the docs caught three spec-vs-reality gaps:

1. **MCP config** lives in `~/.claude.json` (a 118 KB shared file, mode 600) — not `mcp.json` . Makes safe writes the top safety concern.
2. **Plugin enable/disable** is a boolean flip in `settings.json:enabledPlugins` keyed `name@marketplace` — not a key add/remove.
3. **Stop / Notification hooks are not wired** on this box — so the cockpit must install its own state-tracking shim.

What's left

> **▲ TWO MANUAL PASSES BLOCK V1 — ONLY YOU CAN RUN THEM**
>
> There is no API key in the agent environment, so a real Claude session can't be driven headlessly. These two checks need you at the keyboard. Both are confirmations of already-passing spikes, not open risks.

- **D3 visual pass** — eyeball a real Claude TUI rendering in the actual GUI window (fidelity, throughput, detachability).
- **D6 statistical run** — drive a real Claude session through its state transitions to produce the full accuracy numbers.
- **Then: v1 backbone** — the six Pillar-1 features, built behind the now-passed gates, each run through the native-overlap gate.

Step-by-step — how to continue

STEP 1 — D3 VISUAL PASS (~10 MIN)

Open the spike demo and watch a real TUI render through the cockpit bridge:

```
cd ~/Workflows/cc-cockpit/spikes/d3
bash start-demo-session.sh          # spawns private session cockpit-d3, pane %0
cd tauri-app && npm install && npm run dev   # window opens
```

In the pane, run in turn: `vim` (alt-screen + box-drawing), `htop` (colors + live redraw), then a real `claude`. Confirm each renders cleanly. For throughput, run `cat` on a large file in one pane while typing in another. For detachability, from a separate terminal run `tmux -L cockpit-d3 split-window -t d3live -h` and watch the cockpit reflect it.

Teardown when done:

```
tmux -L cockpit-d3 kill-server
```

If any *correct* VT stream visibly corrupts, that's the one spec hard-fail — tell me and we reopen the Tauri decision. Nothing in the spike points there.

STEP 2 — D6 STATISTICAL RUN (~20 MIN)

Drive a real Claude session through its states (working → needs-input → idle → done → killed), several reps, so the parser's accuracy and latency get real numbers. The harness and parser are in `~/Workflows/cc-cockpit/spikes/d6/` (`run_spike.sh`, `parse_state.sh`, `FINDINGS.md`). Capture one real "needs input" prompt so its exact wording is on record.

STEP 3 — GREEN-LIGHT V1

Message me **"build v1 backbone"**. I'll route the six Pillar-1 features to the dev-team behind the passed gates, applying the native-overlap gate to each (P1-F2 / F4 / F5 are flagged thin-wrapper-risk vs native Claude Desktop — they get justified or cut).

> **◆ BOTTOM LINE**
>
> Spec final, both hard gates green, repo clean. Two short manual passes from you close out v1's prerequisites — then the backbone build starts. Say "build v1 backbone" when ready.

REFERENCE — WHERE EVERYTHING LIVES

```
docs/plans/2026-06-18-cc-cockpit-spec.md     # spec spine (+ section-*.md, + -spec-state.md resume point)
~/Workflows/cc-cockpit/spikes/{d6,d3}/       # spikes + FINDINGS
```